Competitive Programming Notebook

py.Sandu

Contents

1 Geometry

1.1 Cross Product

```
double cross_product(Point O, Point A, Point B)
{
    return (A.x - O.x) * (B.y - O.y)
           - (A.y - O.y) * (B.x - O.x);
}
```

1.2 Convex Hull

```
// Returns a list of points on the convex hull
// in counter-clockwise order
vector<Point> convex_hull(vector<Point> A)
{
    int n = A.size(), k = 0;

    if (n <= 3)
        return A;

    vector<Point> ans(2 * n);

    // Sort points lexicographically
    sort(A.begin(), A.end());

    // Build lower hull
    for (int i = 0; i < n; ++i) {

        // If the point at K-1 position is not a part
        // of hull as vector from ans[k-2] to ans[k
    -1]
        // and ans[k-2] to A[i] has a clockwise turn
        while (
            k >= 2
            && cross_product(ans[k - 2], ans[k - 1],
    A[i])
                   <= 0)
            k--;
        ans[k++] = A[i];
    }

    // Build upper hull
    for (size_t i = n - 1, t = k + 1; i > 0; --i) {

        // If the point at K-1 position is not a part
        // of hull as vector from ans[k-2] to ans[k
    -1]
        // and ans[k-2] to A[i] has a clockwise turn
        while (k >= t
               && cross_product(ans[k - 2], ans[k -
    1],
                                A[i - 1])
                      <= 0)
            k--;
        ans[k++] = A[i - 1];
    }

    // Resize the array to desired size
    ans.resize(k - 1);

    return ans;
}
```

1.3 Isleft

```
// Check if a point is to the left of a segment
    defined by two points
bool isLeft(const Point& p, const Point& v1, const
    Point& v2) {
    return (v2.x - v1.x) * (p.y - v1.y) - (v2.y - v1.
    y) * (p.x - v1.x) > 0;
}
```

1.4 Winding Number

```
// Check if a point is inside a polygon using the
    winding number algorithm

bool isInsidePolygon(const Point& p, const vector<
    Point>& polygon) {
    int windingNumber = 0;
    int n = polygon.size();

    for (int i = 0; i < n; i++) {
        const Point& v1 = polygon[i];
        const Point& v2 = polygon[(i + 1) % n];

        if (v1.y <= p.y) {
            if (v2.y > p.y && isLeft(p, v1, v2)) {
                ++windingNumber;
            }
        } else {
            if (v2.y <= p.y && !isLeft(p, v1, v2)) {
                --windingNumber;
            }
        }
    }

    return windingNumber != 0;
}
```

1.5 Point

```
struct Point {
    double x, y;
};
```

2 DS

2.1 Union Find

```
int find(vector<int>& union_find, int a) {
    vector<int> path;

    while(union_find[a] >= 0){
        path.push_back(a);
        a = union_find[a];
    }

    for(int vertice : path)
        union_find[vertice] = a;

    return a;
}

void unite(vector<int>& union_find, int a, int b) {
    a = find(union_find, a);
    b = find(union_find, b);

    if(a == b) return;

    if(abs(union_find[a]) < abs(union_find[b])) swap(
    a, b);

    union_find[a]+=union_find[b];
    union_find[b] = a;
}

// Example usage for Hamming distance problem

class Solution {
public:
```

```
    int minimumHammingDistance(vector<int>& source,
    vector<int>& target, vector<vector<int>>&
    allowedSwaps) {
        vector<int> union_find(source.size(), -1);
        unordered_map<int, unordered_map<int, int>>
    count;
        for(int i = 0; i < allowedSwaps.size(); i++){
            int a = allowedSwaps[i][0];
            int b = allowedSwaps[i][1];
            unite(union_find, a, b);
        }
        for(int i = 0; i < source.size(); i++){
            int group = find(union_find, i);
            count[group][source[i]]++;
            count[group][target[i]]--;
        }
        int ans = 0;
        for(auto& pair : count)
            for(auto& freq : pair.second)
                ans+=abs(freq.second);
        return ans/2;
    }
};
```

2.2 Trie

```
// class for a node of the trie
class TrieNode {
public:
    bool endofWord;
    TrieNode* children[26];

    // Constructore to initialize a trie node
    TrieNode()
    {
        endofWord = false;
        for (int i = 0; i < 26; i++) {
            children[i] = nullptr;
        }
    }
};

// class for the Trie implementation
class Trie {
private:
    TrieNode* root;

public:
    Trie() { root = new TrieNode(); }

    // Function to insert a word into the trie
    void insert(string word)
    {
        TrieNode* node = root;
        for (char c : word) {
            int index = c - 'a';
            if (!node->children[index]) {
                node->children[index] = new TrieNode
    ();
            }
            node = node->children[index];
        }
        node->endofWord = true;
    }

    // Function to search a word in the trie
    bool search(string word)
    {
        TrieNode* node = root;
        for (char c : word) {
            int index = c - 'a';
            if (!node->children[index]) {
                return false;
            }
            node = node->children[index];
        }
        return node->endofWord;
    }

    // Function to check if there is any word in the
    trie
    // that starts with the given prefix
    bool startsWith(string prefix)
    {
        TrieNode* node = root;
        for (char c : prefix) {
            int index = c - 'a';
            if (!node->children[index]) {
                return false;
            }
            node = node->children[index];
        }
        return true;
    }

    // Function to delete a word from the trie
    void deleteWord(string word)
    {
        TrieNode* node = root;
        for (char c : word) {
            int index = c - 'a';
            if (!node->children[index]) {
                return;
            }
            node = node->children[index];
        }
        if (node->endofWord == true) {
            node->endofWord = false;
        }
    }

    // Function to print the trie
    void print(TrieNode* node, string prefix) const
    {
        if (node->endofWord) {
            cout << prefix << endl;
        }
        for (int i = 0; i < 26; i++) {
            if (node->children[i]) {
                print(node->children[i],
                      prefix + char('a' + i));
            }
        }
    }

    // Function to start printing from the root
    void print() const { print(root, ""); }
};
```

2.3 Fenwick Tree

```
// Fenwick Tree to efficiently calculate and update
    prefix sums of an array of values.
// Time complexity: O(log N) for queries and updates

int sum(int i) {
  int sum = 0;
  while(i > 0) {
    sum += T[i];
    i -= i & -i; // flip the last bit
  }
  return sum;
}

void add(int i, int k) {
  while (i < T.size()) {
```

```
    T[i] += k;
    i += i & -i; // add last set bit
  }
}

vi make(vi arr) {
  vi tree = arr;

  for(int i = 0 ; i < tree.size() ; i++) {
    int p = i + (i & -i) // index to parent
    if (p < t.size()) {
        tree[p] = tree[p] + tree[i];
      }
  }
  return tree;
}
```

2.4 Monotonic Stack

```
// Increasing Monotonic Stack
// Time Complexity: O(n)
// Space Complexity: O(n)

vector<int> nextGreaterElements(vector<int>& arr) {
    int n = arr.size();
    vector<int> res(n, -1);
    stack<int> st;

    for (int i = 0; i < 2 * n; i++) {
        int num = arr[i % n];
        while (!st.empty() && arr[st.top()] < num) {
            res[st.top()] = num;
            st.pop();
        }
        if (i < n) {
            st.push(i);
        }
    }

    return res;
}

// Decreasing Monotonic Stack
// Time Complexity: O(n)
// Space Complexity: O(n)

vector<int> nextSmallerElements(vector<int>& arr) {
    int n = arr.size();
    vector<int> res(n, -1);
    stack<int> st;

    for (int i = 0; i < 2 * n; ++i) {
        int num = arr[i % n];
        while (!st.empty() && arr[st.top()] > num) {
            res[st.top()] = num;
            st.pop();
        }
        if (i < n) {
            st.push(i);
        }
    }

    return res;
}
```

2.5 Seg Tree

```
// Segment Tree implementation
// Time complexity: O(log n) for updates and queries

template<typename T>
class SegmentTree {
private:
    int n;
    vector<T> tree;
    T combine(T a, T b) {
        // Combine function for the segment tree (e.g., sum, min, max)
        return a + b; // Example: sum
    }
    void build(vector<T>& arr, int v, int tl, int tr) {
        if (tl == tr) {
            tree[v] = arr[tl];
        } else {
            int tm = (tl + tr) / 2;
            build(arr, v * 2, tl, tm);
            build(arr, v * 2 + 1, tm + 1, tr);
            tree[v] = combine(tree[v * 2], tree[v * 2 + 1]);
        }
    }
    void update(int v, int tl, int tr, int pos, T new_val) {
        if (tl == tr) {
            tree[v] = new_val;
        } else {
            int tm = (tl + tr) / 2;
            if (pos <= tm) {
                update(v * 2, tl, tm, pos, new_val);
            } else {
                update(v * 2 + 1, tm + 1, tr, pos, new_val);
            }
            tree[v] = combine(tree[v * 2], tree[v * 2 + 1]);
        }
    }
    T query(int v, int tl, int tr, int l, int r) {
        if (l > r) {
            return T(); // Return identity element (e.g., 0 for sum)
        }
        if (l == tl && r == tr) {
            return tree[v];
        }
        int tm = (tl + tr) / 2;
        return combine(query(v * 2, tl, tm, l, min(r, tm)),
                       query(v * 2 + 1, tm + 1, tr, max(l, tm + 1), r));
    }
public:
    SegmentTree(vector<T>& arr) {
        n = arr.size();
        tree.resize(4 * n);
        build(arr, 1, 0, n - 1);
    }
    void update(int pos, T new_val) {
        update(1, 0, n - 1, pos, new_val);
    }
    T query(int l, int r) {
        return query(1, 0, n - 1, l, r);
    }
};
```

3 General

4 String

4.1 Kmp

```cpp
// Find all occurrences of a pattern in a given text using the Knuth-Morris-Pratt (KMP) algorithm
// Time complexity: O(n + m) where n is the length of the text and m is the length of the pattern

vector<int> LPS(const string& pattern) {
    int m = pattern.size();
    vector<int> lps(m, 0);
    int len = 0; // length of the previous longest prefix suffix
    int i = 1;

    while (i < m) {
        if (pattern[i] == pattern[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
    return lps;
}

vector<int> KMP(const string& text, const string& pattern) {
    vector<int> lps = LPS(pattern);
    vector<int> occurrences;
    int n = text.size();
    int m = pattern.size();
    int i = 0; // index for text
    int j = 0; // index for pattern

    while (i < n) {
        if (text[i] == pattern[j]) {
            i++;
            j++;
        }
        if (j == m) {
            occurrences.push_back(i - j); // Pattern found at index (i - j)
            j = lps[j - 1]; // Continue to search for the next match
        } else if (i < n && text[i] != pattern[j]) {
            if (j != 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }
    return occurrences;
}
```

5 Math

5.1 Siege Of Eratosthenes

```cpp
// Find all prime numbers up to a given limit using the Sieve of Eratosthenes algorithm
// Time complexity: O(n log log n) where n is the limit

vector<int> findPrimes(int limit) {
    vector<bool> isPrime(limit + 1, true);
    isPrime[0] = isPrime[1] = false;

    for (int i = 2; i * i <= limit; i++) {
        if (isPrime[i]) {
            for (int j = i * i; j <= limit; j += i) {
                isPrime[j] = false;
            }
        }
    }

    vector<int> primes;
    for (int i = 2; i <= limit; i++) {
        if (isPrime[i]) {
            primes.push_back(i);
        }
    }
    return primes;
}
```

5.2 Matrix Fibonacci

```cpp
// Calculate the Nth Fibonacci number using matrix exponentiation.
// Time Complexity: O(log N)
// Space Complexity: O(1)

void matrixExponentiation(vector<vector<long long>>& matrix, int power) {
    int n = matrix.size();
    vector<vector<long long>> result(n, vector<long long>(n, 0));

    // Initialize result as the identity matrix
    for (int i = 0; i < n; i++) {
        result[i][i] = 1;
    }

    while (power) {
        if (power % 2 == 1) {
            matrixMultiplication(result, matrix, result);
        }
        matrixMultiplication(matrix, matrix, matrix);
        power /= 2;
    }

    matrix = result;
}
```

5.3 Matrix Multiplication

```cpp
// Multiply two matrices A and B, and return the result in matrix C.
// Time Complexity: O(n^3)
// Space Complexity: O(n^2)

vector<vector<int>> matrixMultiplication(const vector<vector<int>>& A, const vector<vector<int>>& B) {
    int n = A.size();
    vector<vector<int>> C(n, vector<int>(n, 0)); // Initialize result matrix with zeros

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < n; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
    return C;
}
```

5.4 Solve Linear System

```cpp
// Solve a system of linear equations using LU decomposition
// Time complexity: O(n^3) for decomposition, O(n^2) for forward and backward substitution

void luDecomposition(const vector<vector<double>>& A, vector<vector<double>>& L, vector<vector<double>>& U) {
    int n = A.size();
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i <= j) {
                U[i][j] = A[i][j];
                for (int k = 0; k < i; k++) {
                    U[i][j] -= L[i][k] * U[k][j];
                }
            } else {
                L[i][j] = A[i][j];
                for (int k = 0; k < j; k++) {
                    L[i][j] -= L[i][k] * U[k][j];
                }
                L[i][j] /= U[j][j];
            }
        }
    }
}

void forwardSubstitution(const vector<vector<double>>& L, const vector<double>& b, vector<double>& y) {
    int n = L.size();
    for (int i = 0; i < n; i++) {
        y[i] = b[i];
        for (int j = 0; j < i; j++) {
            y[i] -= L[i][j] * y[j];
        }
    }
}


void backwardSubstitution(const vector<vector<double>>& U, const vector<double>& y, vector<double>& x) {
    int n = U.size();
    for (int i = n - 1; i >= 0; i--) {
        x[i] = y[i];
        for (int j = i + 1; j < n; j++) {
            x[i] -= U[i][j] * x[j];
        }
        x[i] /= U[i][i];
    }
}
```

5.5 Modular Exponentiation

```cpp
// Calculates (x^n) % MOD
// Time Complexity: O(log n)
// Space Complexity: O(1)

int powMod(int x, int n, int MOD){
    int res = 1;

    while(n >= 1){
        if(n & 1){
            res = (res * x) % MOD;
            n--;
        }
        else{
            x = (x * x) % MOD;
            n /= 2;
        }
    }
    return res;
}
```

5.6 Spf

```cpp
// SPF sieve implementation for finding the smallest prime factor of each number up to a given limit.
// Time complexity: O(n log log n) for the sieve construction, O(log n) for factorization of each number.

#define MAXN 1000000 // Maximum limit for the sieve
std::vector<int> spf(MAXN + 1);

void sieve() {
    for (int i = 1; i <= MAXN; i++) {
        spf[i] = i;
    }
    for (int i = 2; i * i <= MAXN; i++) {
        if (spf[i] == i) {
            for (int j = i * i; j <= MAXN; j += i) {
                if (spf[j] == j) {
                    spf[j] = i;
                }
            }
        }
    }
}
```

5.7 Gcd

```cpp
// Calculates the Greatest Common Divisor (GCD) of two integers using Euclidean algorithm
// Time Complexity: O(log(min(a, b)))
// Space Complexity: O(log(min(a, b)))

int gcd(int a, int b){
    if (a == 0) return b;
    return gcd(b % a, a);
}
```

5.8 Extended Euclidean Algorithm

```cpp
// Extended Euclidean Algorithm
// Returns (gcd(a, b), x, y) such that a*x + b*y = gcd(a, b)

int extendedEuclidean(int a, int b, int& x, int& y){
    if(a == 0){
        x = 0;
        y = 1;
        return b;
    }
    int x1, y1;
    int gcd = extendedEuclidean(b % a, a, x1, y1);
    x = y1 - (b / a) * x1;
    y = x1;
    return gcd;
}
```

6 Graph

6.1 Bellman-ford

```cpp
// Find the shortest path from a source vertex to all other vertices in a graph with negative weight edges and detect negative weight cycles.
// Time Complexity: O(V * E)
// Space Complexity: O(V)

void bellmanFord(Graph& graph, int src) {
    int V = graph.size();
    vector<int> dist(V, INT_MAX);
```

```
    dist[src] = 0;

    for (int i = 1; i < V; i++) {
        for (int u = 0; u < V; u++) {
            for (int v = 0; v < V; v++) {
                if (graph[u][v] != 0 && dist[u] != INT_MAX && dist[u] + graph[u][v] < dist[v]) {
                    dist[v] = dist[u] + graph[u][v];
                }
            }
        }
    }

    // Check for negative weight cycles
    for (int u = 0; u < V; u++) {
        for (int v = 0; v < V; v++) {
            if (graph[u][v] != 0 && dist[u] != INT_MAX && dist[u] + graph[u][v] < dist[v]) {
                cout << "Graph contains negative weight cycle" << endl;
                return;
            }
        }
    }
}
```

6.2 Floyd-warshall

```
// Find the shortest paths in a weighted graph with positive or negative edge weights (but with no negative cycles).
// Time Complexity: O(V^3)
// Space Complexity: O(V^2)

void floydWarshall(vector<vector<int>> &dist) {
    int V = dist.size();
    int INF = 1e8;

    for (int k = 0; k < V; k++) {

        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if(dist[i][k] != INF && dist[k][j]!= INF )
                    dist[i][j] = min(dist[i][j],
                                     dist[i][k] + dist[k][j]);
            }
        }
    }
}
```

6.3 Ford-fulkerson

```
// Find maximum flow in graph
// Time complexity: O(|V| * EËĘ2)
// Space complexity: O(V)
bool bfs(int residual[V][V], int start, int target, vi parent){
    vector<bool> visited(V, false);

    queue<int> q;
    q.push(start);
    visited[start] = true;
    parent[start] = -1;

    while(!q.empty()){
        int u = q.front();
        q.pop();

        for(int v = 0 ; v <= V ; v++){
            if(!visited[v] and residual[u][v] > 0){
                if(v == target){
                    parent[v] = u;
                    return true;
                }
                q.push(v);
                parent[v] = u;
                visited[v] = true;
            }
        }
    }
    return false;
}

int fordFulkerson(int g[V][V], int start, int target){
    int u, v;
    int residual[V][V];

    for(u = 0 ; u < V ; u++)
        for(v = 0 ; v < V ; v++)
            residual[u][v] = g[u][v];

    vi parent(V);
    int maxFlow = 0;

    while(bfs(residual, start, target, parent)){
        int pathFlow = INT_MAX;
        for(v = target ; v != start ; v = parent[v]){
            u = parent[v];
            pathFlow = min(pathFlow, residual[u][v]);
        }

        for(v = target ; v != start ; v = parent[v]){
            u = parent[v];
            residual[u][v] -= pathFlow;
            residual[v][u] += pathFlow;
        }
        maxFlow += pathFlow;
    }
    return maxFlow;
}
```

6.4 Mst Prim

```
// Prim's algorithm for finding the Minimum Spanning Tree
// Time complexity: O(E log V) where E is the number of edges and V is the number of vertices

struct Edge {
    int u, v, weight;
    bool operator<(const Edge& other) const {
        return weight > other.weight; // Min-heap
    }
};

vector<Edge> primMST(const vector<vector<int>>& graph) {
    int n = graph.size();
    vector<bool> visited(n, false);
    vector<Edge> mst;
    priority_queue<Edge> pq;

    // Start with the first vertex
    visited[0] = true;
    for (int i = 0; i < n; ++i) {
        if (graph[0][i] != 0) {
            pq.push({0, i, graph[0][i]});
        }
    }

    while (!pq.empty()) {
        Edge current = pq.top();
```

```
        pq.pop();

        if (visited[current.v]) continue;

        visited[current.v] = true;
        mst.push_back(current);

        for (int i = 0; i < n; ++i) {
            if (!visited[i] && graph[current.v][i] != 0) {
                pq.push({current.v, i, graph[current.v][i]});
            }
        }
    }

    return mst;
}
```

6.5 Dfs

```
// Depth First Search (DFS)
// Time Complexity: O(V + E)
// Space Complexity: O(V)

// Recursive DFS
void dfs(const Graph& graph, int vertex, vector<bool>& visited) {
    visited[vertex] = true;

    for (int neighbor : graph[vertex]) {
        if (!visited[neighbor]) {
            dfs(graph, neighbor, visited);
        }
    }
}

// Iterative DFS
void dfsIterative(const Graph& graph, int start) {
    int V = graph.size();
    vector<bool> visited(V, false);
    stack<int> s;
    s.push(start);

    while (!s.empty()) {
        int vertex = s.top();
        s.pop();

        if (!visited[vertex]) {
            visited[vertex] = true;

            for (int neighbor : graph[vertex]) {
                if (!visited[neighbor]) {
                    s.push(neighbor);
                }
            }
        }
    }
```

6.6 Bfs

```
// Breadth First Search (BFS)
// Time Complexity: O(V + E)
// Space Complexity: O(V)
void bfs(const Graph& graph, int start) {
    int V = graph.size();
    vector<bool> visited(V, false);
    queue<int> q;

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int vertex = q.front();
        q.pop();
        cout << vertex << " ";

        for (int neighbor : graph[vertex]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}
```

6.7 Reverse Graph

```
// Reverse the edges of a directed graph
// Time Complexity: O(V + E)
// Space Complexity: O(V + E)

Graph reverseGraph(const Graph& graph) {
    int V = graph.size();
    Graph reversedGraph(V);

    for (int u = 0; u < V; u++) {
        for (int v : graph[u]) {
            reversedGraph[v].push_back(u);
        }
    }

    return reversedGraph;
}
```

6.8 Eulerean Path

```
// Find the Eulerian path in a graph using Hierholzer's algorithm
// Time complexity: O(E) where E is the number of edges in the graph

void findEulerianPath(const vector<vector<int>>& graph, int start, vector<int>& path) {
    stack<int> s;
    s.push(start);

    while (!s.empty()) {
        int v = s.top();
        if (graph[v].empty()) {
            path.push_back(v);
            s.pop();
        } else {
            int u = graph[v].back();
            graph[v].pop_back(); // Remove the edge from v to u
            s.push(u);
        }
    }
}
```

6.9 Djikstra

```
// Find shortest path from a source vertex to all other vertices in a graph with non-negative edge weights
// Time Complexity: O((V + E) log V)
// Space Complexity: O(V)

void dijkstra(const Graph& graph, int src) {
    int V = graph.size();
    vector<int> dist(V, INT_MAX);
    dist[src] = 0;

```

```
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq; // Min-heap priority queue
    pq.push({0, src});

    while (!pq.empty()) {
        int u = pq.top().second;
        pq.pop();

        for (const auto& adj : graph[u]) {
            int v = adj.first;
            int weight = adj.second;

            if (dist[u] != INT_MAX && dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                pq.push({dist[v], v});
            }
        }
    }
}
```

7 DP

7.1 Knapsack

```
// Knapsack problem implementation using dynamic programming
// Time complexity: O(n*w), where n is the number of items and w is the maximum weight
// Space complexity: O(n*w) for the dp table

int knapsack(int w, vector<ii>& items){
    int n = items.size();
    vector<vi> dp(n+1, vi(w+1, 0));

    for(int i = 1 ; i <= n ; i++){
        for(int j = 0 ; j <= w ; j++){
            if(items[i-1].first > j) dp[i][j] = dp[i-1][j];
            else dp[i][j] = max(dp[i-1][j], dp[i-1][j-items[i-1].first] + items[i-1].second);
        }
    }

    return dp[n][w];
}
```

7.2 Coins Change

```
// Find the minimum number of coins needed to make a given amount using a set of coin denominations
// Time complexity: O(n * m) where n is the amount and m is the number of coin denominations

int coinChange(const vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, INT_MAX);
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (const int& coin : coins) {
            if (coin <= i && dp[i - coin] != INT_MAX) {
                dp[i] = min(dp[i], dp[i - coin] + 1);
            }
        }
    }

    return dp[amount] == INT_MAX ? -1 : dp[amount];
}
```

7.3 Longest Increasing Subsequence

```
// Find the length of the longest increasing subsequence in a given array
// Time complexity: O(n log n) where n is the length of the input array

int lis(vector<int>& arr) {
    vector<int> dp;
    for (int num : arr) {
        auto it = lower_bound(dp.begin(), dp.end(), num);
        if (it == dp.end()) {
            dp.push_back(num);
        } else {
            *it = num;
        }
    }
    return dp.size();
}
```

7.4 Kadane Algorithm

```
// Find maximum subarray sum using Kadane's algorithm
// Time complexity: O(n) where n is the length of the input array

int maxSubArraySum(const vector<int>& arr) {
    int maxSum = arr[0];
    int maxLocal = arr[0];

    for (int i = 0; i < arr.size(); i++) {
        maxLocal = max(arr[i], maxLocal + arr[i]);
        maxSum = max(maxSum, maxLocal);
    }

    return maxSum;
}
```